ASP.NET MVC

MVC Core

Eng. Basma Hussien

MVC Filters

MVC Filters

- In ASP.NET MVC, a user request is routed to the appropriate controller and action method. However, there may be circumstances where you want to execute some logic before or after an action method executes. ASP.NET MVC provides filters for this purpose.

- ASP.NET MVC Filter is a custom class where you can write custom logic to execute before or after an action method executes. Filters can be applied to an action method or controller

MVC Filters (Cont.)

- Filters can be applied to an action method or controller
- You could apply filters in a declarative or programmatic way:
  - Declarative means by applying a filter attribute to an action method or controller class
  - Programmatic means by implementing a corresponding interface or abstract class

MVC Filters Types

- The ASP.NET MVC framework supports four different types of filters:
  - **Authorization Filters** – Implements the IAuthorizationFilter attribute.
  - **Action Filters** – Implements the IActionFilter attribute.
  - **Result Filters** – Implements the IResultFilter attribute.
  - **Exception Filters** – Implements the IExceptionFilter attribute.
- Filters are executed in the order listed above. For example, authorization filters are always executed before action filters and exception filters are always executed after every other type of filter.
- Authorization filters are used to implement authentication and authorization for controller actions.

MVC Filters Types

- MVC provides different types of filters. The following table list filter types, built-in filters, and interface that must be implemented to create custom filters.

Filter Type	Description	Built-in Filter	Interface
Authorization filters	Performs authentication and authorizes before executing an action method.	[Authorize]	IAuthorizationFilter
Action filters	Performs some operation before and after an action method executes.	[ActionName]	IActionFilter
Result filters	Performs some operation before or after the execution of the view.	[OutputCache]	IResultFilter
Exception filters	Performs some operation if there is an unhandled exception thrown during the execution of the ASP.NET MVC pipeline.	[HandleError]	IExceptionFilter

Conventional routing
& Attribute Routing

Routing

- In ASP.NET Core, routing is handled by routing middleware, which matches the URLs of incoming requests to actions or other endpoints.

- Controller actions are either conventionally routed or attribute-routed.

- Conventional routing is similar to the route table approach used in ASP.NET MVC and Web API. Whether you're using conventional, attribute, or both, you need to configure your app to use the routing middleware.

- To use the middleware, add the following code to your *Program.cs* file:
  - app.UseRouting();

Conventional Routing

Conventional Routing

- With conventional routing, you set up one or more conventions that will be used to match incoming URLs to *endpoints* in the app.

- In ASP.NET Core, these endpoints may be controller actions, like in ASP.NET MVC or Web API.

- In addition to (UseRouting), this configuration specifies a default routing convention, which is the fairly standard {controller}/{action}/{id?} pattern that's been recommended since the first versions of ASP.NET MVC:
- app.UseEndpoints(endpoints =>

```
{
        endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
});
```

Attribute Routing

Attribute Routing

- Apply attribute routing on any action:

- //...../Order/GetOrders/3
- //...../customers/3/Orders

```
[Route("customers/{customerId}/orders")]
public ActionResult GetOrders(int customerId) {…}
```

RoutePrefix

- RoutePrefix on a controller:

  [Route(“Company/Finance")]

- Then on each action in controller you should specify a specific route:

  [Route("customers/{id}")]

  [Route("")]

Override RoutePrefix

Use a tilde **(~)** on the method attribute to override the route prefix:

```
[Route("mvc/books")]
public class BooksController : Controller
{
    // GET /mvc/authors/1/books
    [Route("~/mvc/authors/{authorId:int}/books")]
    public IEnumerable<Book> GetByAuthor(int authorId) { ... }
}
```

Route Constraints

- To apply Route Constraints:

```
[Route("users/{id:int:min(10)}")]
public User GetUserById(int id) { ... }
```

Constraint	Description	Example
Alpha	Matches uppercase or lowercase Latin alphabet characters (a-z, A-Z)	{x:alpha}
Bool	Matches a Boolean value.	{x:bool}
Datetime	Matches a **DateTime** value.	{x:datetime
Decimal	Matches a decimal value.	{x:decimal}
Double	Matches a 64-bit floating-point value.	{x:double}
Float	Matches a 32-bit floating-point value.	{x:float}
Guid	Matches a GUID value.	{x:guid}
Int	Matches a 32-bit integer value.	{x:int}

Length	Matches a string with the specified length or within a specified range of lengths.	{x:length(6)} {x:length(1,20)}
Long	Matches a 64-bit integer value.	{x:long}
Max	Matches an integer with a maximum value.	{x:max(10)}
maxlength	Matches a string with a maximum length.	{x:maxlength(10)}
Min	Matches an integer with a minimum value.	{x:min(10)}
minlength	Matches a string with a minimum length.	{x:minlength(10)}
Range	Matches an integer within a range of values.	{x:range(10,50)}
Regex	Matches a regular expression.	{x:regex(^\d{3}-\d{3}-\d{4}$)}

Membership & Identity Authentication

ASP.NET Membership

- It was designed to solve site membership requirements that were common in 2005, which involved Forms Authentication, and a SQL Server database for user names, passwords, and profile data.

- Today there is a much broader array of data storage options for web applications, and most developers want to enable their sites to use social identity providers for authentication and authorization functionality.

ASP.NET Membership Limitations

- The database schema was designed for SQL Server and you can't change it.
- Membership database is limited to describe users identity.
- Since the log-in/log-out functionality is based on Forms Authentication, the membership system can't use OWIN.

OWIN

- OWIN includes middleware components for authentication, including support for log-ins using external identity providers (like Microsoft Accounts, Facebook, Google, Twitter), and log-ins using organizational accounts from on-premises Active Directory or Azure Active Directory.

- OWIN also includes support for OAuth 2.0, JWT and CORS.

ASP.NET Identity

ASP.NET Identity was developed with the following goals:

- One ASP.NET Identity system
- Persistence control
- Claims Based
- Role provider
- Social Login Providers
- OWIN Integration
- Unit testability

One ASP.NET Identity system

- ASP.NET Identity can be used with all of the ASP.NET frameworks, such as ASP.NET MVC, Web Forms, Web Pages, Web API, and SignalR.
- ASP.NET Identity can be used when you are building web, phone, store, or hybrid applications.

Persistence control

- By default, the ASP.NET Identity system stores all the user information in a database. ASP.NET Identity uses ***Entity Framework Code First*** to implement all of its persistence mechanism.

- Since you control the database schema, common tasks such as changing table names or changing the data type of primary keys is simple to do.

- It's easy to plug in different storage mechanisms such as SharePoint, Azure Storage Table Service, NoSQL databases, etc., without having to throw System.NotImplementedExceptions exceptions.

Claims Based

- ASP.NET Identity supports claims-based authentication, where the user's identity is represented as a set of claims.
- Ease of plugging in profile data about the user, You have control over the schema of user and profile information.
- Claims allow developers to be a lot more expressive in describing a user's identity than roles allow.
- For more info about ClaimTypes: https://docs.microsoft.com/en-us/dotnet/api/system.security.claims.claimtypes?view=net-5.0

Social Login Providers

- You can easily add social log-ins such as Microsoft Account, Facebook, Twitter, Google, and others to your application, and store the user-specific data in your application.

Demo

Validate MVC App Using External Logins

Live Accounts Authentication

- You should Choose "***Individual User***" Authentication MVC template when creating your web application
- You have to create a project on "Google.developer.console" or what so ever provider you will use for authentication process delegation
- You need a "***Client ID & Client Secret***" to be able to use provider API for authenticating your web application

Demo